Advanced Unsupervised Learning Algorithms

Prepared by: Joseph Bakarji

nature

Explore content | About the journal | Publish with us

nature > review articles > article

Review Article | Published: 27 May 2015

Deep learning

Yann LeCun, Yoshua Bengio & Geoffrey Hinton

Nature **521**, 436–444 (2015) | Cite this article

https://www.nature.com/articles/nature14539

The future of deep learning

Unsupervised learning[91,92,93,94,95,96,97,98] had a catalytic effect in reviving interest in deep learning, but has since been overshadowed by the successes of purely supervised learning. Although we have not focused on it in this Review, we expect unsupervised learning to become far more important in the longer term. Human and animal learning is largely unsupervised: we discover the structure of the world by observing it, not by being told the name of every object.

Natural language understanding is another area in which deep learning is poised to make a large impact over the next few years. We expect systems that use RNNs to understand sentences or whole documents will become much better when they learn strategies for selectively attending to one part at a time[76,86].

Geoffrey Hinton, Yoshua Bengio sign statement urging suspension of AGI development

BY MARIA DEUTSCHER

Hundreds of scientists, politicians, entrepreneurs and artists have signed a statement urging the suspension of efforts to develop artificial general intelligence, or AGI.

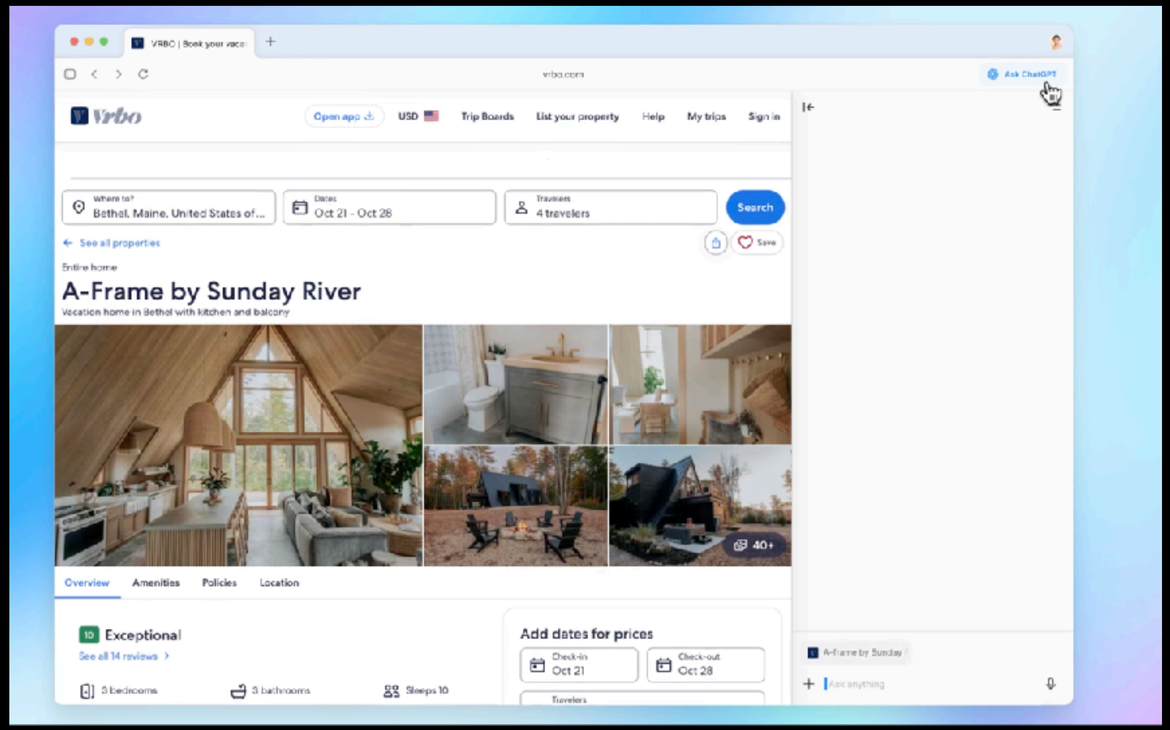
OpenAI releases Atlas Browser

OpenAI

February 24, 2023 Safety

Planning for AGI and beyond

Our mission is to ensure that artificial general intelligence—AI systems that are generally smarter than humans—benefits all of humanity.

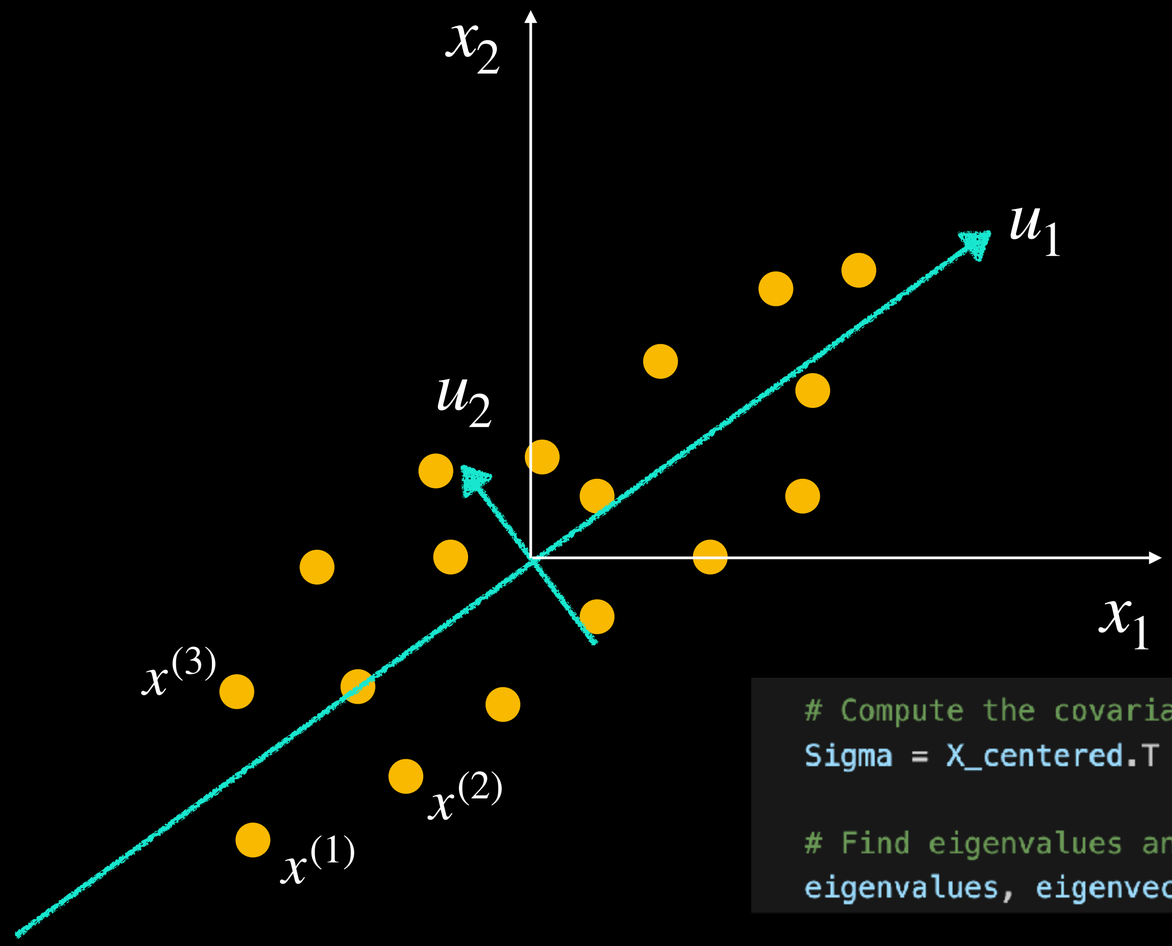


Dimensionality Reduction

Principal Component Analysis

Dataset

x_1	x_2
1.2	1.2
3.2	5.4
4.3	6.4
3.2	5.4
...	...




```
# Compute the covariance matrix
Sigma = X_centered.T @ X_centered

# Find eigenvalues and eigenvectors of the covariance matrix
eigenvalues, eigenvectors = np.linalg.eig(Sigma)
```

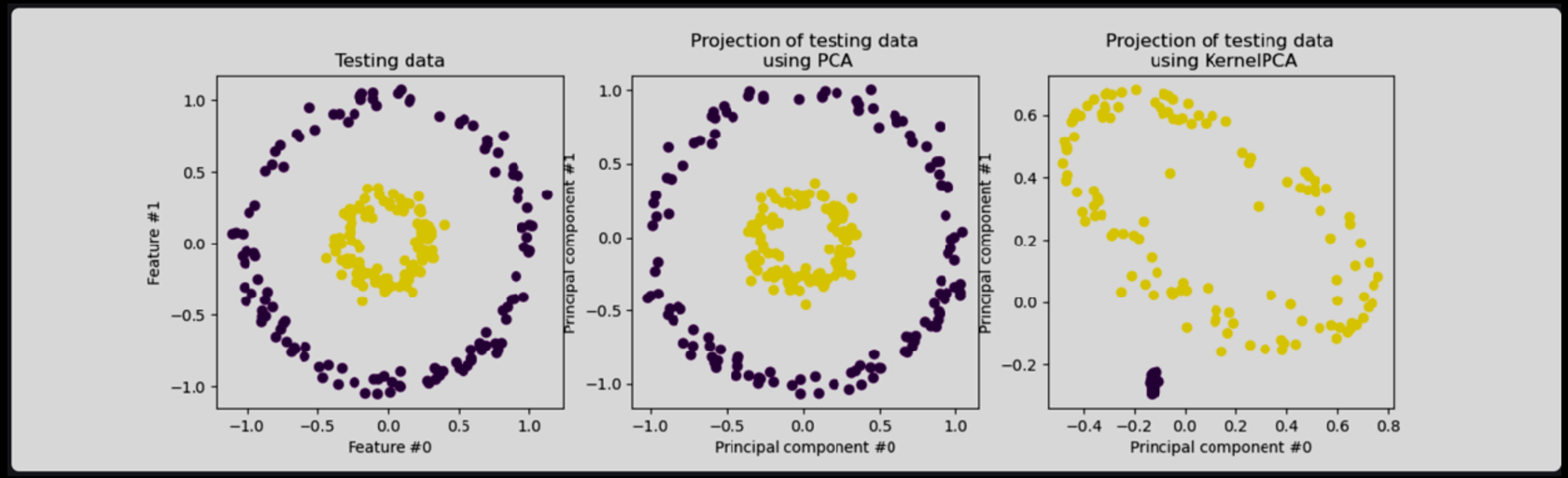
or `U, S, Vt = np.linalg.svd(data_centered)`

Kernel PCA

Extension of PCA which achieves non-linear dimensionality reduction through the use of kernels

Dataset

x_1	x_2
1.2	1.2
3.2	5.4
4.3	6.4
3.2	5.4
...	...



Define a nonlinear feature space through a kernel

$$\phi(x) = \begin{bmatrix} x_1 \\ x_2 \\ x_1^2 \\ x_1 x_2 \\ \vdots \end{bmatrix}$$

→

Perform PCA on the new space

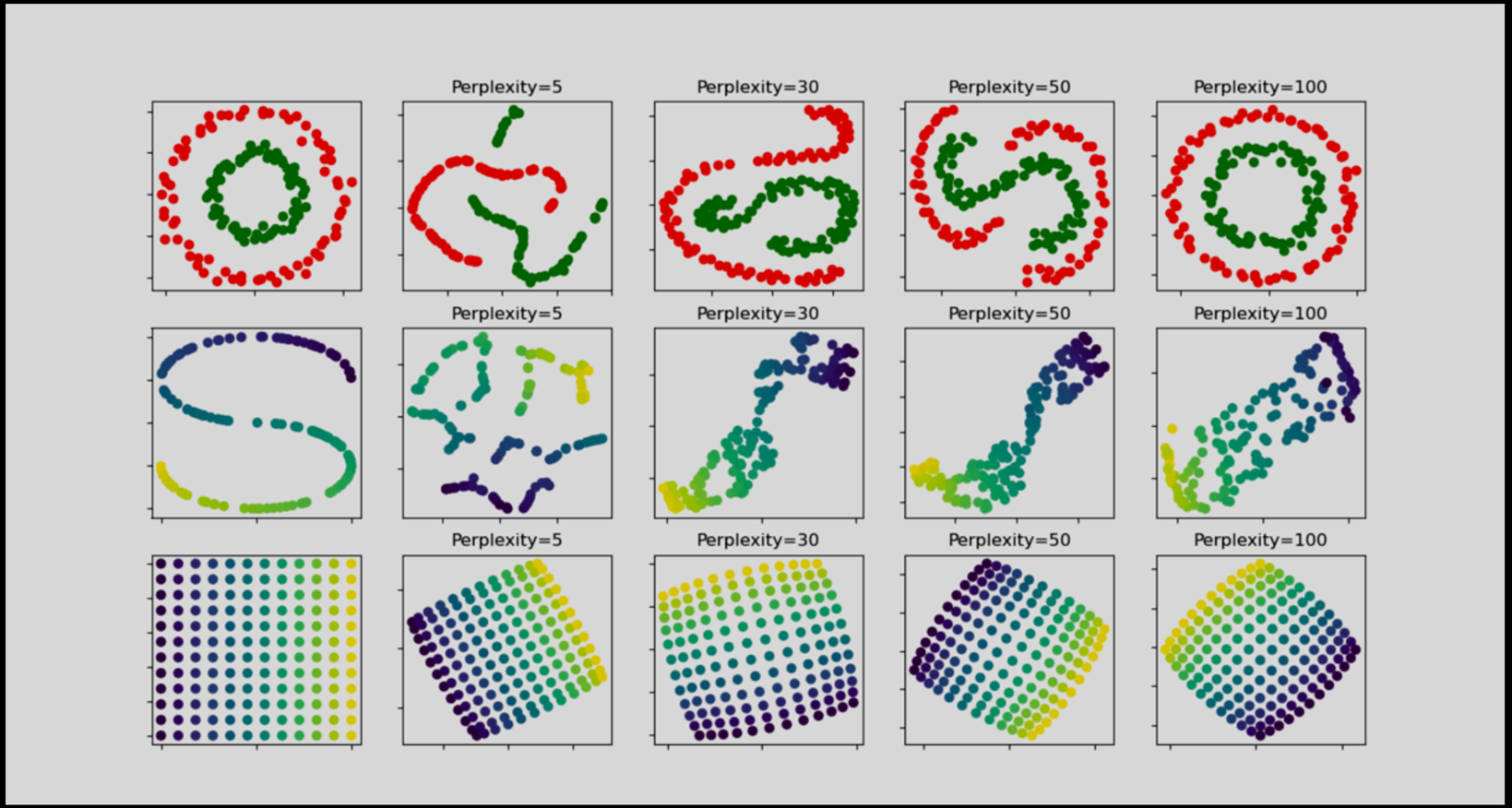
Dimensionality Reduction

t-Distributed Stochastic Neighbor Embedding (t-SNE)

Represents high-dimensional data in a lower-dimensional space while preserving the relationships between data points

Dataset

x_1	x_2
1.2	1.2
3.2	5.4
4.3	6.4
3.2	5.4
...	...



https://scikit-learn.org/1.5/auto_examples/manifold/plot_t_sne_perplexity.html#sphx-glr-auto-examples-manifold-plot-t-sne-perplexity-py

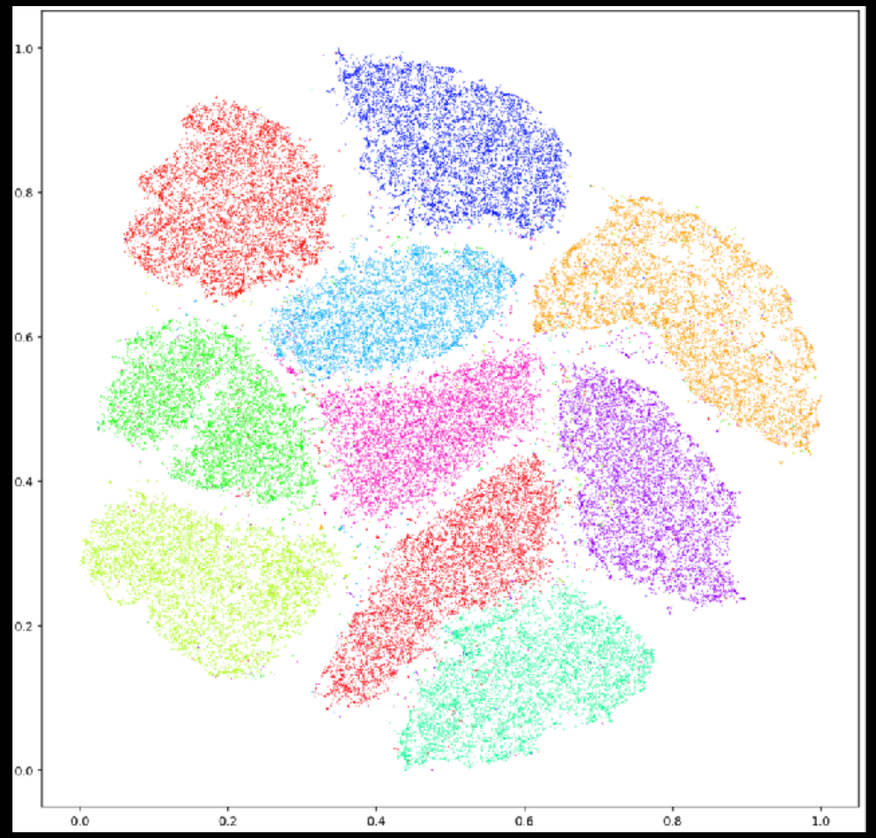
Dimensionality Reduction

t-Distributed Stochastic Neighbor Embedding (t-SNE)

t-SNE of the MNIST data

Dataset

x_1	x_2
1.2	1.2
3.2	5.4
4.3	6.4
3.2	5.4
…	…



Dimensionality Reduction

t-Distributed Stochastic Neighbor Embedding (t-SNE)

Dataset

x_1	x_2
1.2	1.2
3.2	5.4
4.3	6.4
3.2	5.4
…	…

The similarity between two data points, x_i & x_j is calculated using a conditional probability

$$P(x_j \mid x_i) = \frac{\exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma_i^2}\right)}{\sum_{k \neq i} \exp\left(-\frac{\|x_i - x_k\|^2}{2\sigma_i^2}\right)}$$

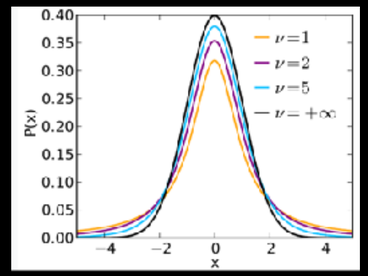
Then a joint distribution $P(x_i, x_j)$ is created by the mean

$$P(x_i, x_j) = [P(x_i \mid x_j) + P(x_j \mid x_i)]/2n$$

In lower-dimensions (2-3D), the joint distribution between points in the reduced space is given by

$$Q(x_i, x_j) = \frac{\left(1 + \|y_i - y_j\|^2\right)^{-1}}{\sum_{k \neq l} \left(1 + \|y_k - y_l\|^2\right)^{-1}}$$

Which is a Student's t-distribution



https://scikit-learn.org/1.5/auto_examples/manifold/plot_t_sne_perplexity.html#sphx-glr-auto-examples-manifold-plot-t-sne-perplexity-py

Dimensionality Reduction

t-Distributed Stochastic Neighbor Embedding (t-SNE)

Dataset

x_1	x_2
1.2	1.2
3.2	5.4
4.3	6.4
3.2	5.4
…	…

$$P(x_j \mid x_i) = \frac{\exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma_i^2}\right)}{\sum_{k \neq i} \exp\left(-\frac{\|x_i - x_k\|^2}{2\sigma_i^2}\right)}$$

$$P(x_i, x_j) = [P(x_i \mid x_j) + P(x_j \mid x_i)]/2n$$

$$Q(x_i, x_j) = \frac{\left(1 + \|y_i - y_j\|^2\right)^{-1}}{\sum_{k \neq l} \left(1 + \|y_k - y_l\|^2\right)^{-1}}$$

“Distance” between them is minimized using gradient descent

$$\mathrm{KL}(P\|Q) = \sum_{i \neq j} P_{ij} \log \frac{P_{ij}}{Q_{ij}}$$

The Kullback-Leibler (KL) divergence is Often used to minimize the distance between two distributions

https://scikit-learn.org/1.5/auto_examples/manifold/plot_t_sne_perplexity.html#sphx-glr-auto-examples-manifold-plot-t-sne-perplexity-py

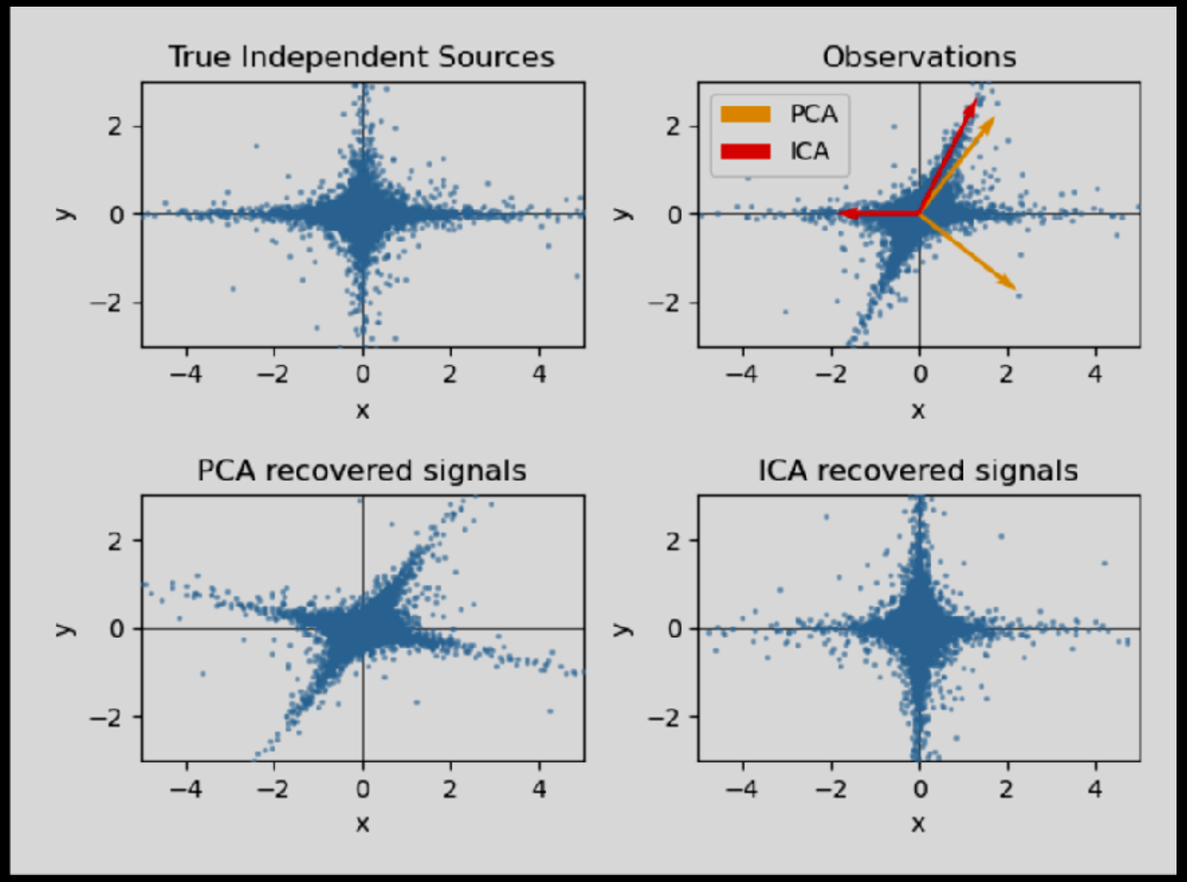
Dimensionality Reduction

Independent Component Analysis (ICA)

The goal of ICA is to express observed data X (A matrix of n observed signals) as a linear combination of statistically independent source signals

Dataset

x_1	x_2
1.2	1.2
3.2	5.4
4.3	6.4
3.2	5.4
…	…



https://scikit-learn.org/stable/auto_examples/decomposition/plot_ica_vs_pca.html#sphx-glr-auto-examples-decomposition-plot-ica-vs-pca-py

Dimensionality Reduction

Independent Component Analysis (ICA)

The goal of ICA is to express observed data X (a matrix of n observed signals) as a linear combination of statistically independent source signals

Dataset

x_1	x_2
1.2	1.2
3.2	5.4
4.3	6.4
3.2	5.4
...	...

The observed data X_i can be represented as a linear combination of the source signals S_j with the mixing matrix elements A_{ij}:

$$X_i = \sum_j A_{ij} S_j$$

This equation states that each observed variable X_i is a sum of contributions from each source S_j, weighted by the mixing coefficients A_{ij}. In matrix form, this can be written compactly as:

$$X = AS$$

where X is the vector of observed variables, A is the mixing matrix, and S is the vector of source signals. The goal of ICA is to find the inverse (or unmixing matrix W) such that:

$$S = WX$$

https://scikit-learn.org/stable/auto_examples/decomposition/plot_ica_vs_pca.html#sphx-glr-auto-examples-decomposition-plot-ica-vs-pca-py

Dimensionality Reduction

Factor Analysis

In unsupervised learning we only have a dataset $X = \{x_1, x_2, \ldots, x_n\}$. How can this dataset be described mathematically? A very simple `continuous latent variable` model for X is

$$x_i = Wh_i + \mu + \epsilon$$

The vector h_i is called "latent" because it is unobserved. ϵ is considered a noise term distributed according to a Gaussian with mean 0 and covariance Ψ (i.e. $\epsilon \sim \mathcal{N}(0, \Psi)$), μ is some arbitrary offset vector. Such a model is called "generative" as it describes how x_i is generated from h_i. If we use all the x_i's as columns to form a matrix $\mathbf{X}$ and all the h_i's as columns of a matrix $\mathbf{H}$ then we can write (with suitably defined $\mathbf{M}$ and $\mathbf{E}$):

$$\mathbf{X} = W\mathbf{H} + \mathbf{M} + \mathbf{E}$$

In other words, we *decomposed* matrix $\mathbf{X}$.

If h_i is given, the above equation automatically implies the following probabilistic interpretation:

$$p(x_i|h_i) = \mathcal{N}(Wh_i + \mu, \Psi)$$

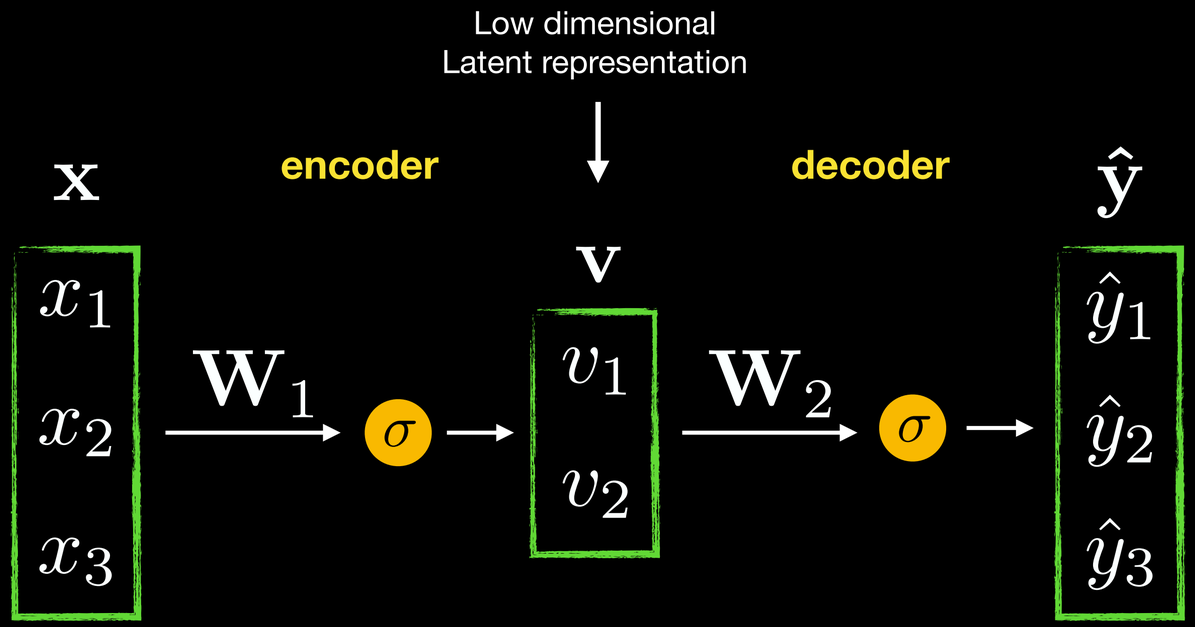
For a complete probabilistic model we also need a prior distribution for the latent variable h. The most straightforward assumption (based on the nice properties of the Gaussian distribution) is $h \sim \mathcal{N}(0, \mathbf{I})$. This yields a Gaussian as the marginal distribution of x:

$$p(x) = \mathcal{N}(\mu, WW^T + \Psi)$$

https://scikit-learn.org/stable/modules/decomposition.html#factor-analysis

Dimensionality Reduction

Neural Networks: Auto-encoders



$$\mathcal{L} = \| f_{\mathbf{W}_1 \mathbf{W}_2}(\mathbf{x}) - \mathbf{x} \|^2$$

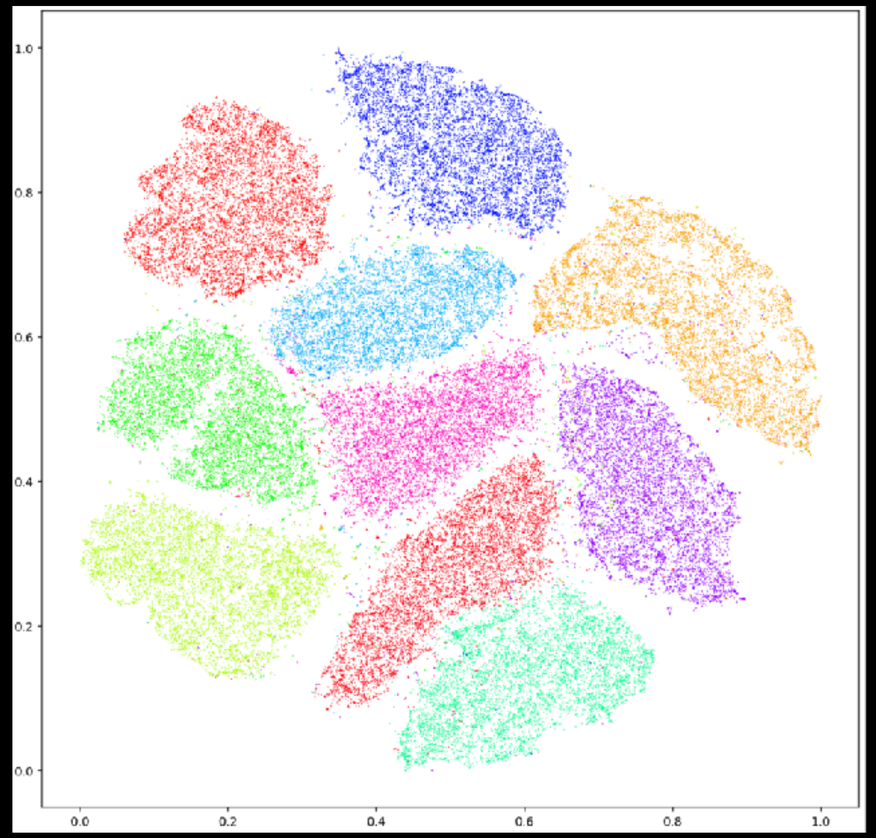
if $\sigma = I$, network reduced to SVD decomposition

$$\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^*$$

Dimensionality Reduction

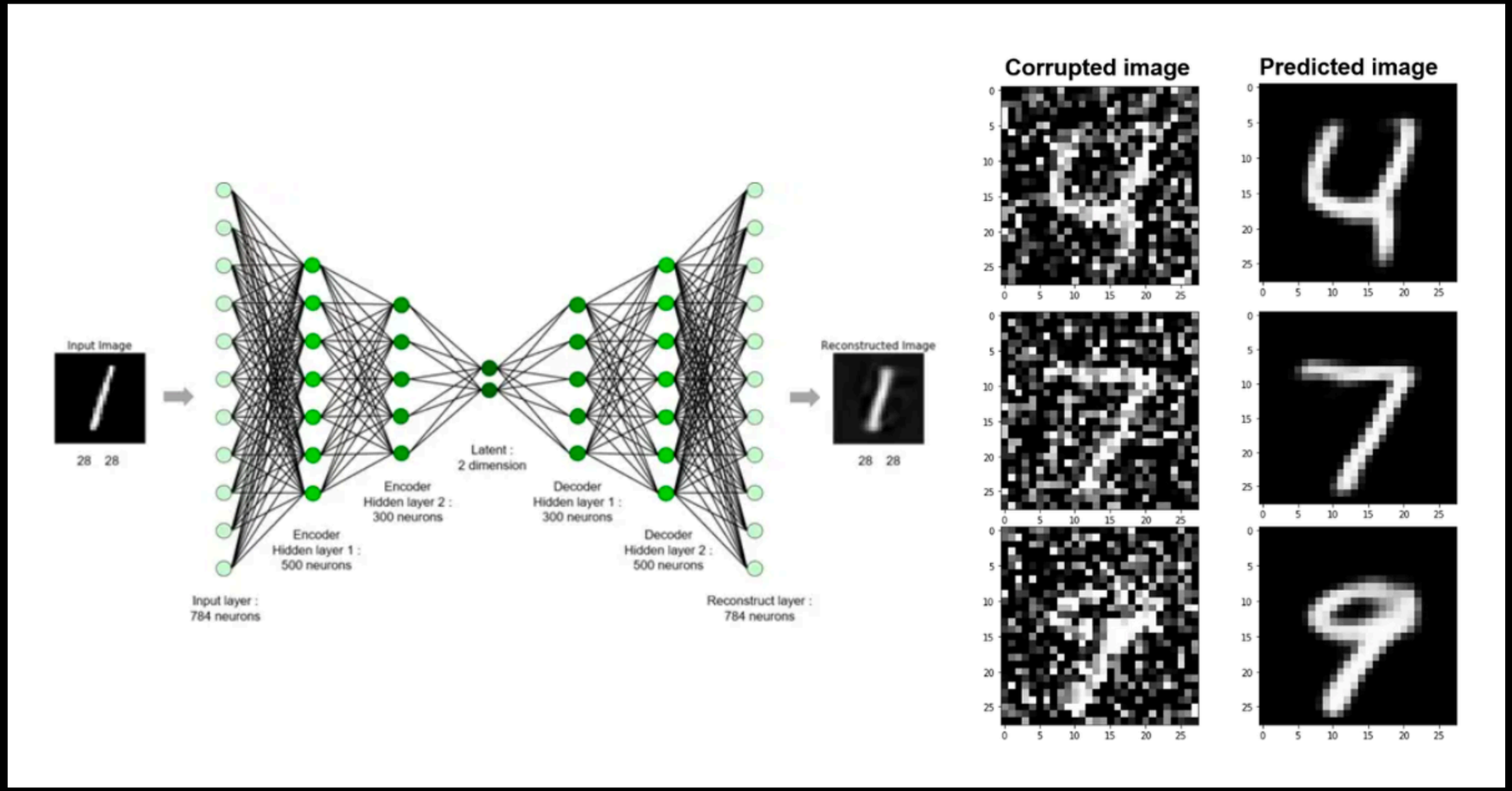
Dataset

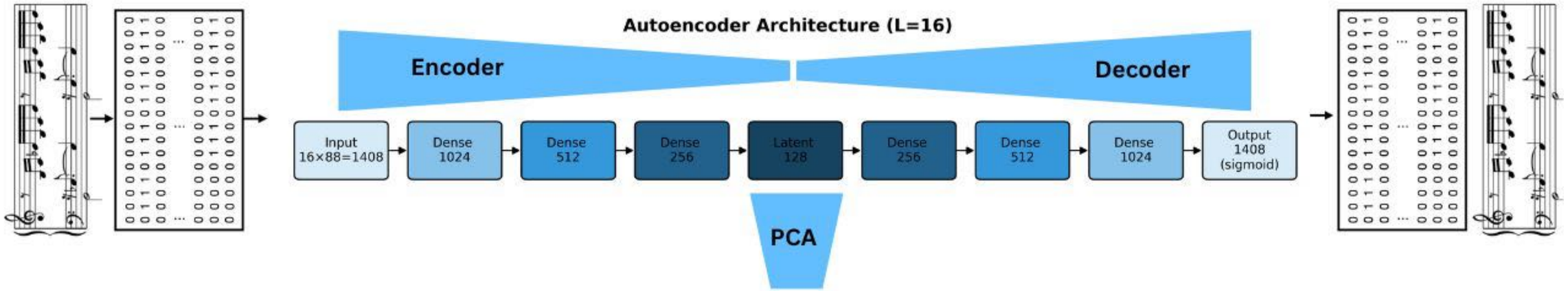
x_1	x_2
1.2	1.2
3.2	5.4
4.3	6.4
3.2	5.4
...	...

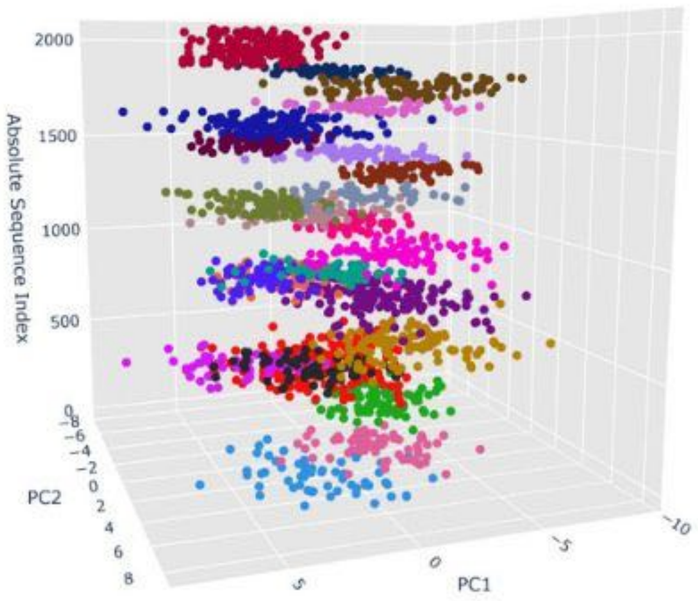


Denoising

Neural Networks: Auto-encoders

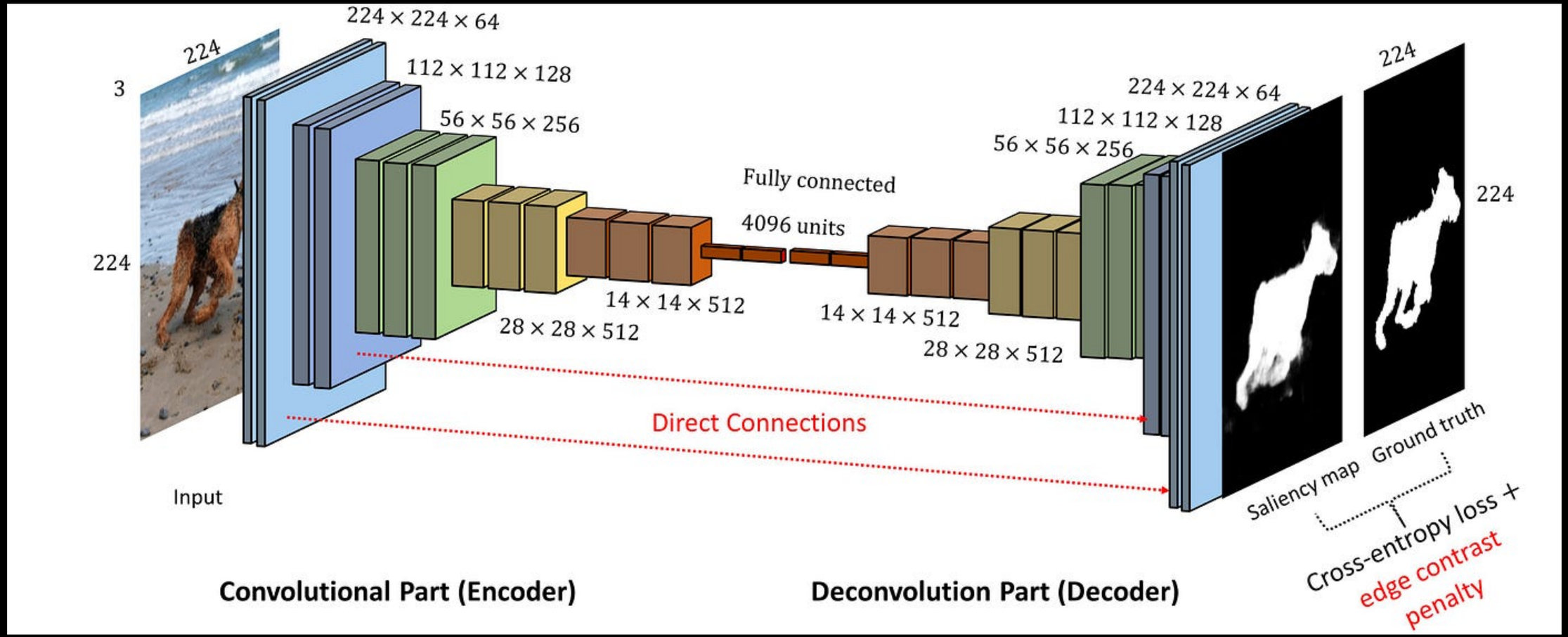



Autoencoder Architecture (L=16)
Encoder
Decoder
Input
16×88=1408
Dense
1024
Dense
512
Dense
256
Latent
128
Dense
256
Dense
512
Dense
1024
Output
1408
(sigmoid)


PCA
Absolute Sequence Index
2000
1500
1000
500
0
PC2
PC1

Najla Sadek

Convolutional Autoencoders

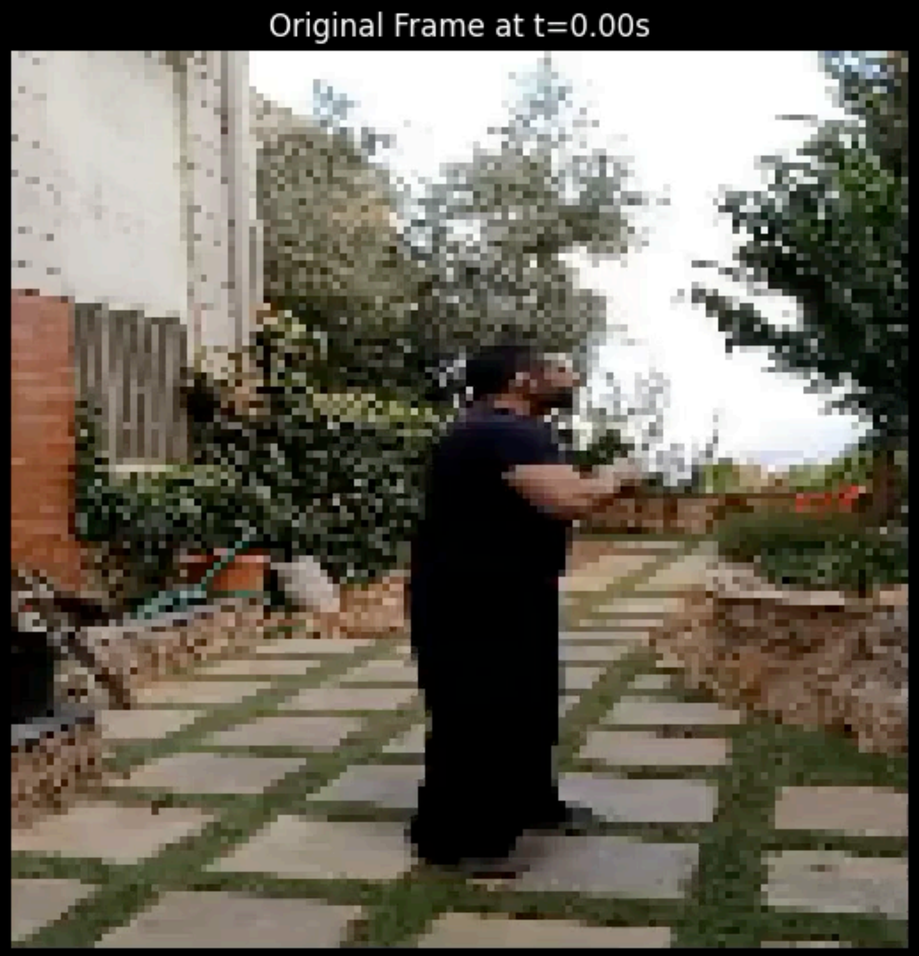


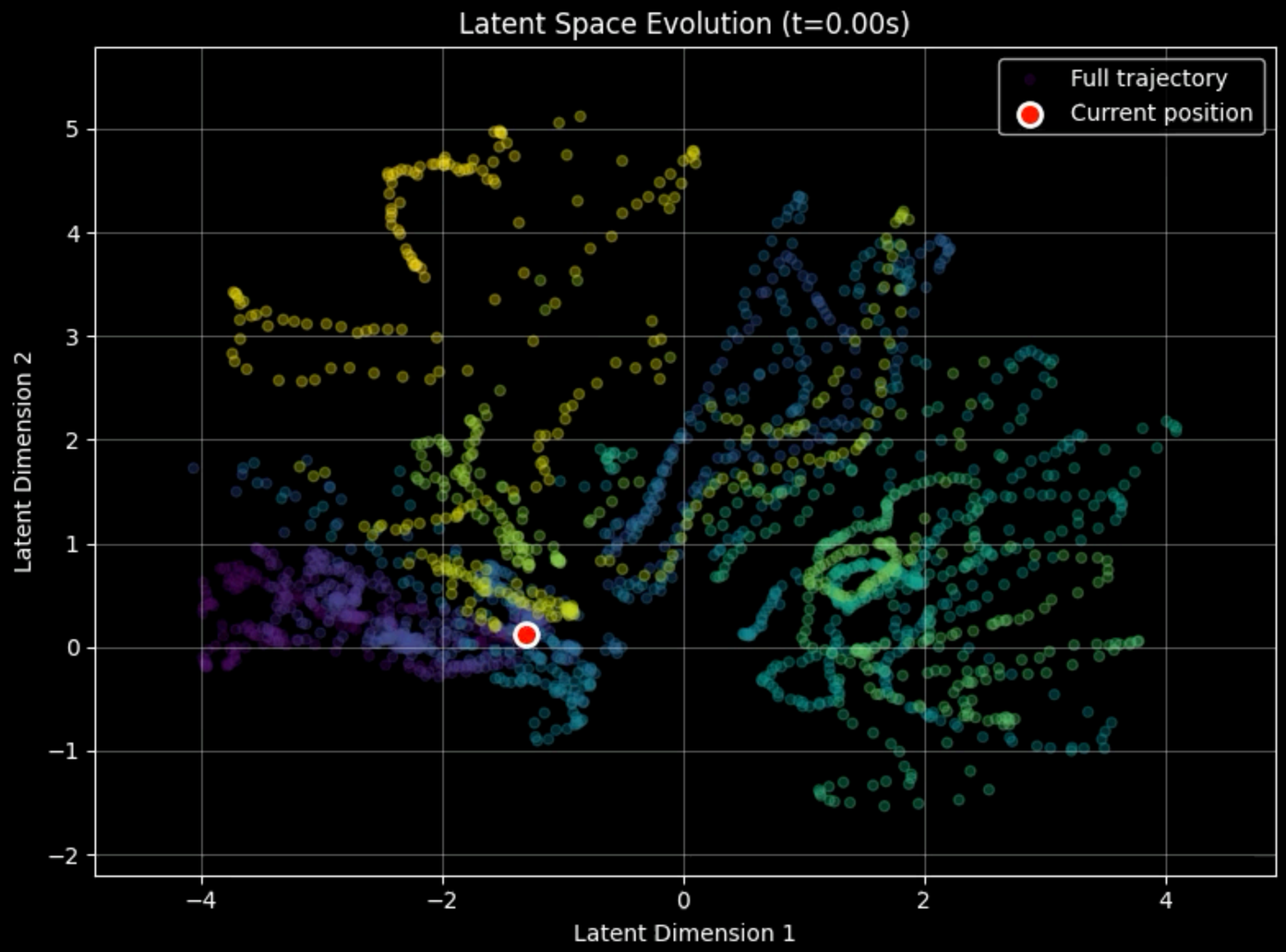
Music Flow

Music Intelligence Lab

Olivier Chiniara

Music Flow

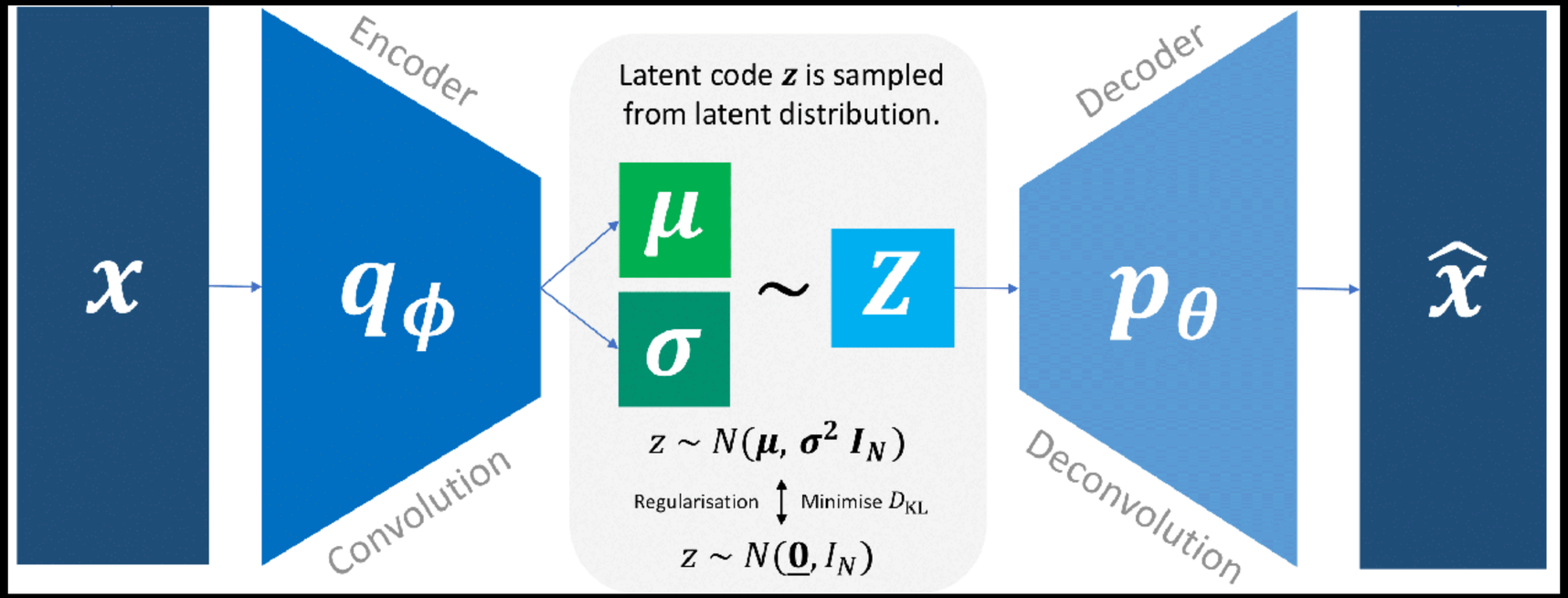

Original Frame at t=0.00s


Latent Space Evolution (t=0.00s)
Full trajectory
Current position
Latent Dimension 2
Latent Dimension 1
5
4
3
2
1
0
−1
−2
−4
−2
0
2
4

```python
class MotionVAE(nn.Module):
    """Variational Autoencoder for motion features"""
    def __init__(self, input_dim, hidden_dim=32, latent_dim=2):
        super(MotionVAE, self).__init__()

        # Encoder
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
        )

        self.fc_mu = nn.Linear(hidden_dim, latent_dim)
        self.fc_logvar = nn.Linear(hidden_dim, latent_dim)

        # Decoder
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, input_dim),
        )

    def encode(self, x):
        h = self.encoder(x)
        mu = self.fc_mu(h)
        logvar = self.fc_logvar(h)
        return mu, logvar

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + eps * std

    def decode(self, z):
        return self.decoder(z)

    def forward(self, x):
        mu, logvar = self.encode(x)
        z = self.reparameterize(mu, logvar)
        recon = self.decode(z)
        return recon, mu, logvar
```

```python
def vae_loss(recon_x, x, mu, logvar, beta=0.1):
    """VAE loss = reconstruction + KL divergence"""
    recon_loss = nn.MSELoss()(recon_x, x)
    kl_loss = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
    kl_loss = kl_loss / x.size(0)  # Normalize by batch size
    return recon_loss + beta * kl_loss
```

Variational Autoencoders



https://github.com/alexjmanlove/convolutional-variational-autoencoders

$$\mathscr{L}(x;\theta,\phi) = -\,\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x\,|\,z)] + D_{\mathrm{KL}}\big(q_\phi(z\,|\,x)\,\|\,p(z)\big)$$

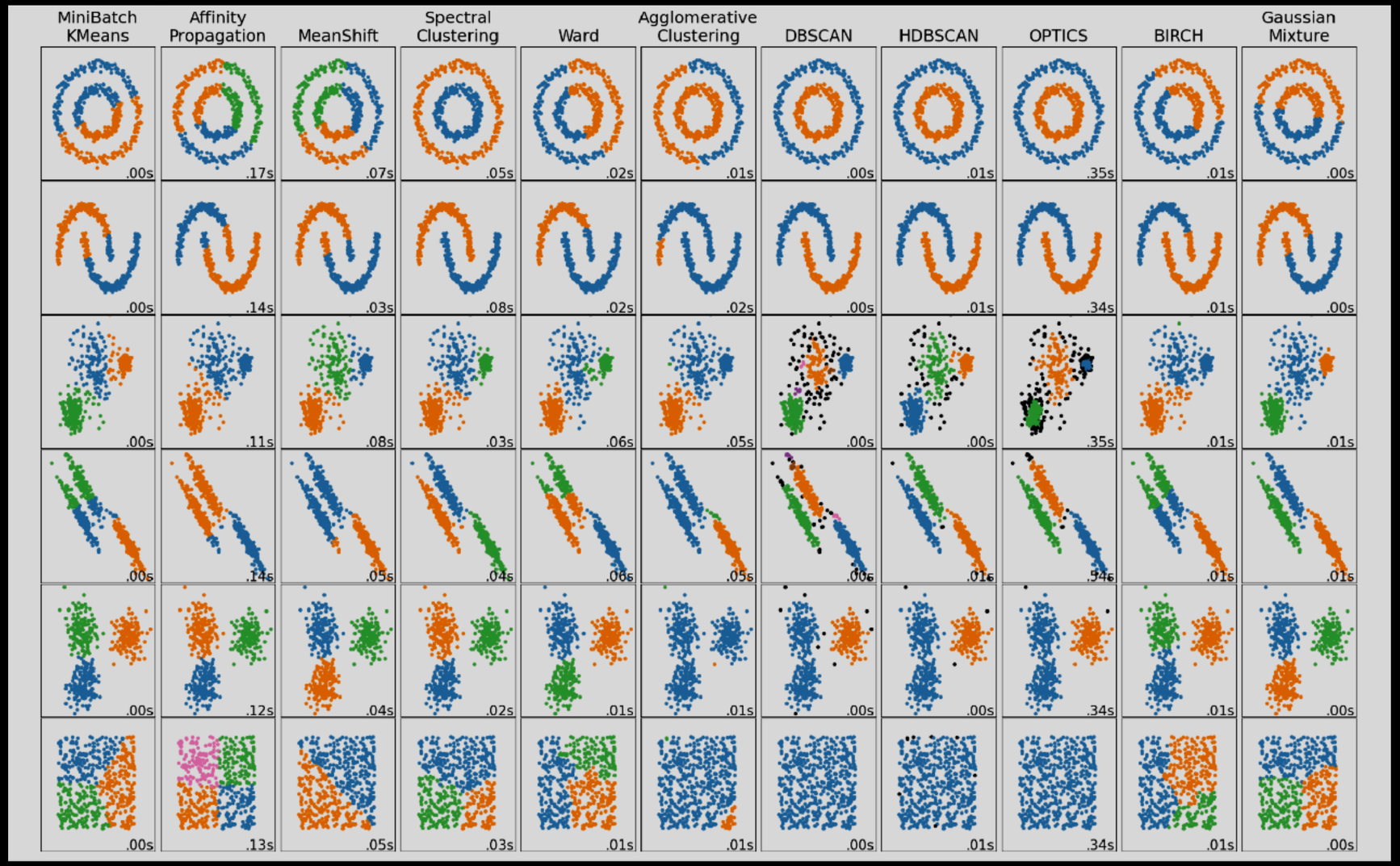
Reconstruction Loss + KL Divergence $\frac{1}{2}\sum_i \left(\mu_i^2 + \sigma_i^2 - \log\sigma_i^2 - 1\right)$

Clustering

A zoo of clustering methods

Dataset

x_1	x_2
1.2	1.2
3.2	5.4
4.3	6.4
3.2	5.4
…	…

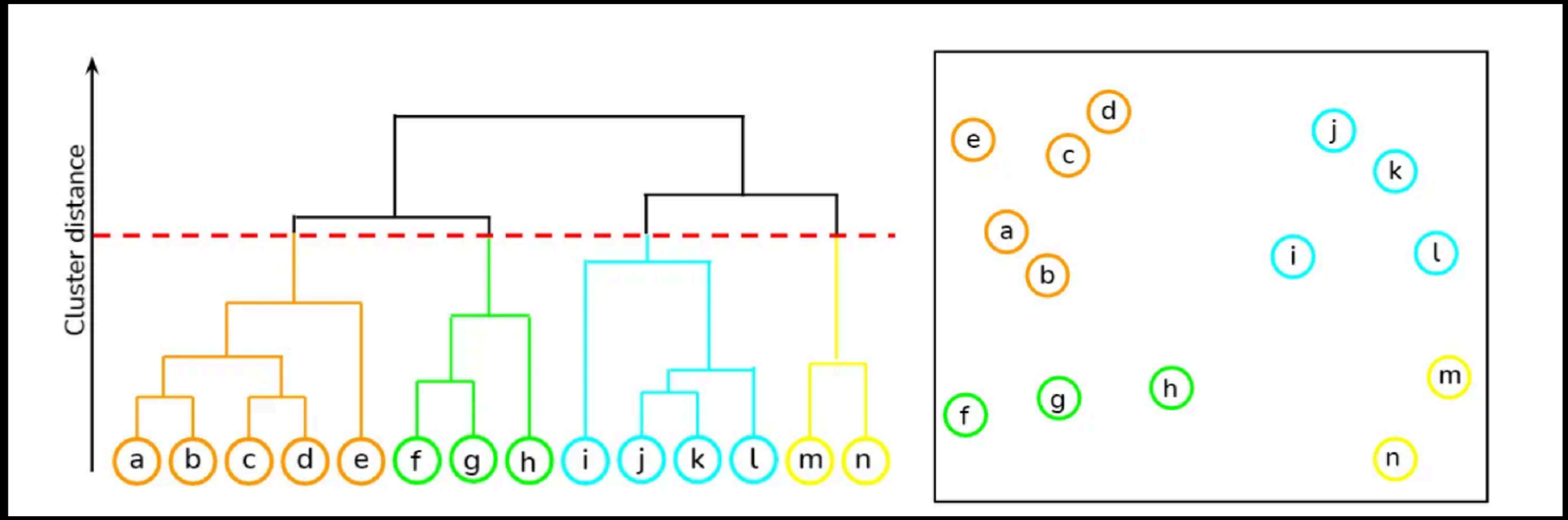


https://scikit-learn.org/stable/modules/clustering.html#hierarchical-clustering

Clustering

Hierarchical Clustering

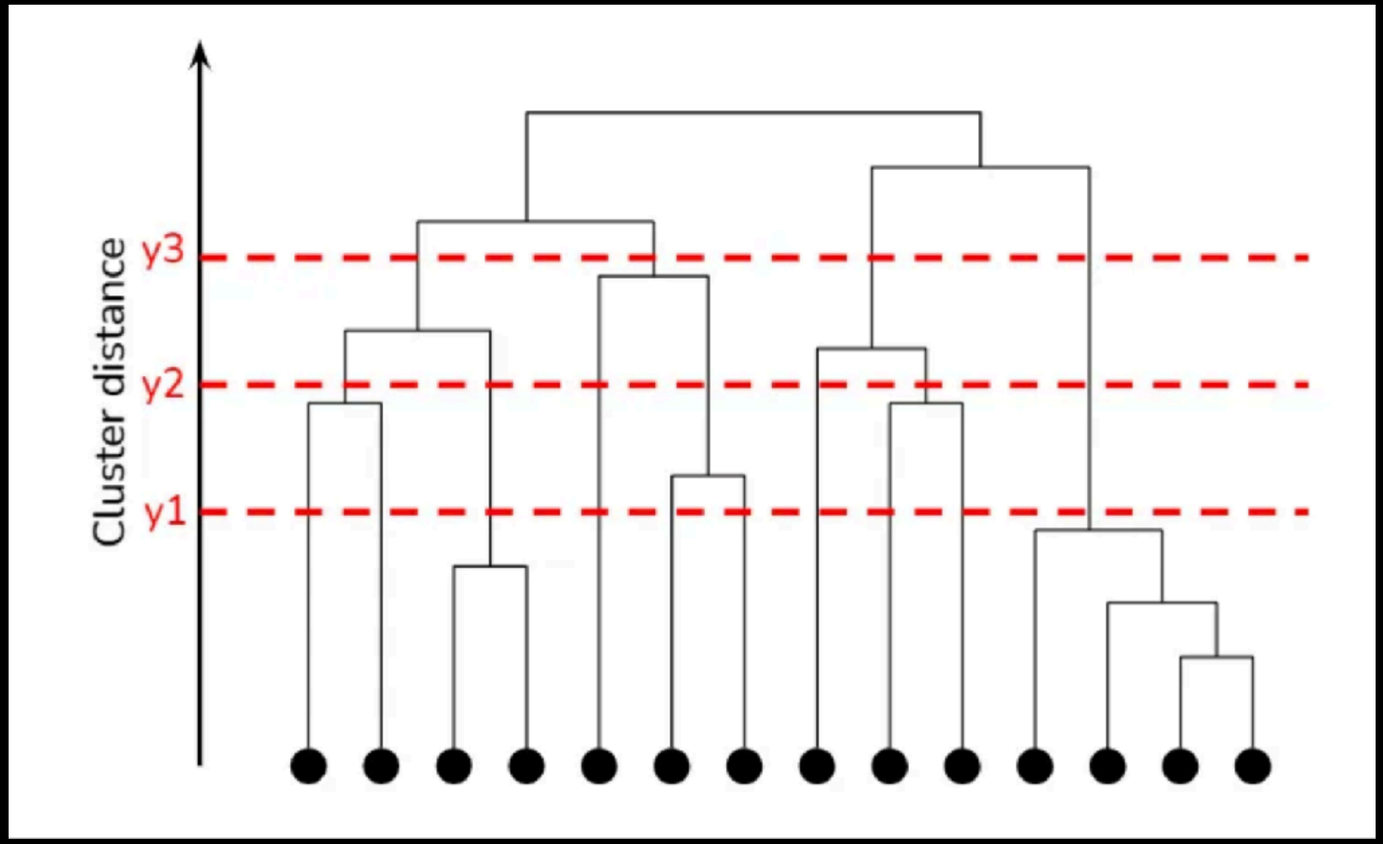
Dendrogram



https://towardsdatascience.com/hierarchical-clustering-explained-e59b13846da8

Clustering

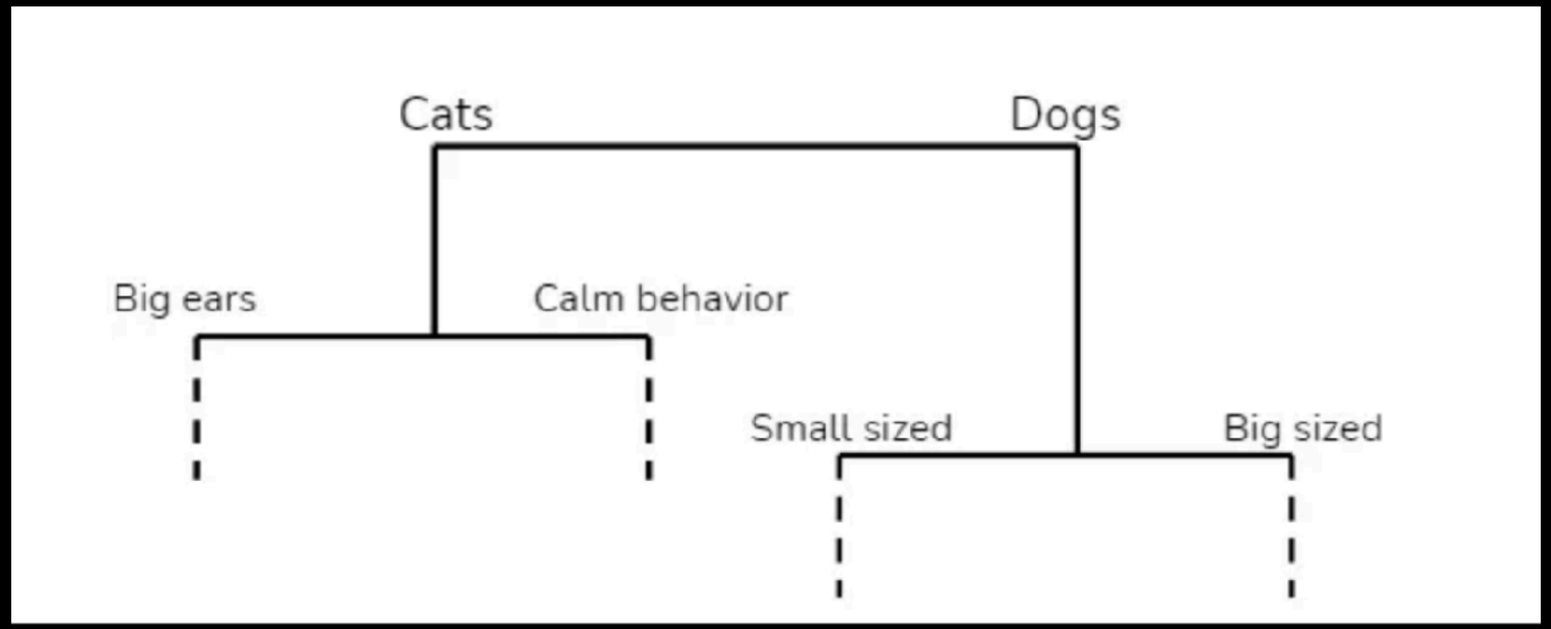
Hierarchical Clustering



https://towardsdatascience.com/hierarchical-clustering-explained-e59b13846da8

Clustering

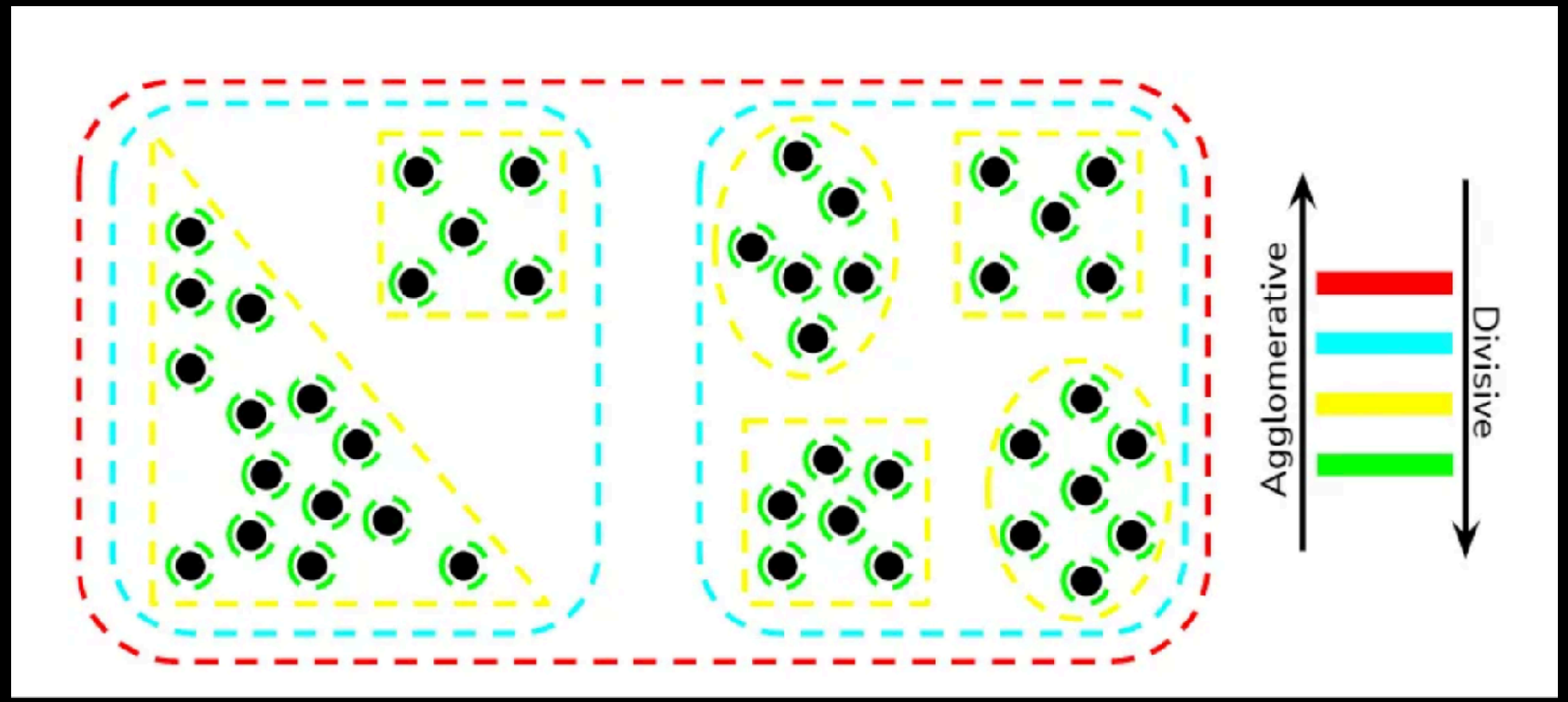
Hierarchical Clustering



https://towardsdatascience.com/hierarchical-clustering-explained-e59b13846da8

Clustering

Hierarchical Clustering



https://towardsdatascience.com/hierarchical-clustering-explained-e59b13846da8

Clustering

Hierarchical Clustering

Objective function for Ward's method

$$\sum_{C} \sum_{x \in C} \|x - \mu_C\|^2$$

- C represents a cluster in the set of all clusters
- x is a data point within cluster C
- μ_c is the centroid (mean) of cluster C
- $\|x - \mu_C\|^2$ is the squared Euclidean distance between a point x and the centroid μ_c

https://towardsdatascience.com/hierarchical-clustering-explained-e59b13846da8